

Computational Lab Material

Introduction

In this computational lab we will be striking a balance between one of the expensive ab initio methods presented in the Lectures and phenomenological techniques: we will be constructing a trial BCS wavefunction by solving the BCS gap equations using a chiral N³LO potential. Here you will find notes and instructions.

The gap equations

We will be solving the gap equations in the 1S_0 channel of neutron matter, that is, the Cooper pairs are $l = 0$ and spin-singlet pairs of neutrons whose binding originates from the 1S_0 channel of the N³LO potential. The gap equations in this channel are

$$\Delta(k) = -\frac{1}{\pi} \int dp p^2 V(k, p) \frac{\Delta(p)}{E(p)} ,$$
$$\rho = \frac{1}{2\pi^2} \int dk k^2 \left[1 - \frac{\xi(k)}{E(k)} \right] ,$$

where $V(k, p)$ is the 1S_0 channel of the potential and

$$\xi(k) = \epsilon(k) - \mu ,$$
$$E(k) = \sqrt{\xi^2(k) + \Delta^2(k)} ,$$

The pairing gap then can be defined in two ways:

$$\bar{\Delta}_F = \Delta(k_F) ,$$

or

$$\Delta_F = \min_k E(k) .$$

We will explore both.

Finally the occupation probabilities $p(k)$ and the condensate amplitudes F_k are

$$p(k) = v^2(k) , \quad F(k) = v(k)u(k) ,$$

with

$$u^2(k) = \frac{1}{2} \left[1 \pm \frac{\xi(k)}{E(k)} \right] .$$

Fortran refresher

We will be using Fortran90. Here's a quick refresher.

A main program must start and end with

```

program first
...
end program first

```

Variables must be declared along with their type (and possibly kind). The statement `implicit none` should come before the variable declaration otherwise the variable's name determines its type as:

- Variable names starting with **i-n** (the first two letters of “integer”) specify *integer* variables
 - All other variables are *real*
- We will (and you should) avoid implicit typing.

If you feel that you need to refresh your fortran here's a short program that reads the diagonal parts of the N^3LO potential on a defined grid from the file `vkk_n3lo.dat` (found [here](#)) and prints the value of the potential at the grid's end:

```

program first
! This is where a `use module` statement would go

implicit none
real(kind=8) :: x1,x2,y
integer :: i
character(len=50) :: print_form

! unformatted write statement
write(*,*) 'Reading N3LO V(k,k)'

! opening a file for reading. status='old' ensures the file is not rewritten
! avoid units 5 and 6 as they are pre-assigned to stdin and stdout
open(unit=1,file='vkk_n3lo.dat',status='old')

! read from file at unit 1
do i=1,3000
    read(1,*) x1,x2,y
end do
close(unit=1)

! formatted write statement
print_form='(A20 f12.8 A24 f12.8)'
print_form=trim(print_form)
write(*,print_form) 'At highest momentum k=',x1, 'the potential is V(k,k)=', y

end program first

```

Aside from mathematical functions, the above program contains all the commands that we will use. In fortran we can also make modules that contain functions and variables. An example of a module is shown in the wrapper for the N^3LO potential below.

gnuplot refresher

We will be using `gnuplot` for fast plotting and interpreting. To plot a file `f.dat` containing data in the form

```
x    y
.    .
.    .
.    .
```

with `gnuplot` using points, using lines, or points and lines, you must use the command(s)

```
plot 'f.dat'
plot 'f.dat' with lines
plot 'f.dat' with linespoints
```

or the abbreviations

```
p 'f.dat'
p 'f.dat' w l
p 'f.dat' w lp
```

You can add set the x- and y-labels as

```
set xlabel 'some-label'
set ylabel 'some-label'
```

If the file has more columns and you want to plots columns 2 and 3 and also add a title the above commands and abbreviations are modified as

```
plot 'f.dat' using 2:3 with linespoint title 'some-name'
p 'f.dat' u 2:3 w lp t 'some-name'
```

Finally, `gnuplot` supports for loops, so that if you want to plot columns 2 and 3 from 3 files `file1.dat` , `file2.dat` , and `file3.dat` you can do so as

```
p for [n=1:3] 'file'.n.'.dat' u 2:3 w lp t 'some-name-'.n.'
```

The potential

We will be using the $N^3\text{LO}$ nucleon-nucleon potential whose circulated form can be found [here](#)

This potential is already formulated in momentum space so we won't need to Fourier transform it.

However the potential's routine is written in fortran77 and must be wrapped to be practical. A practical wrapper is written below:

```

module vn3lo
!wrapper for n3lo450
implicit real(kind=8) (a-h,o-z)
contains

include 'n3lo450.f90'

real(kind=8) function n3lo_1s0(k1,k2)
real(kind=8), intent(in) :: k1, k2
common /crdwrt/ kread,kwrite,kpunch,kda(9)
common /cpot/ v(6),xmev,ymev
common /cstate/ j,heform,sing,trip,coup,endeplabel
common /cnn/ inn
logical heform,sing,trip,coup,endepl
character*4 label
pi = acos(-1d0)
hbarc = 197.327053d0

xmev = k1*hbarc
ymev = k2*hbarc

kread = 5
kwrite = 6
!kpunch
!kda(9)
j=0 !1S[0]
heform = .false.
sing = .true.
trip = .true.
coup = .true.
inn = 3 !neutron-neutron
call n3lo450new
n3lo_1s0 = 0.5*pi*v(1)*hbarc**3d0
end function n3lo_1s0
end module vn3lo

```

The grid

The integration of the gap equations will be done using Legendre quadrature, that is, the integrals will be discretized on Legendre nodes and evaluated as weighted sums. A function that generates the corresponding nodes and weights can be found [here](#).

Solution for a single chemical potential

As a **first step**, we will generate the potential in the 1S_0 channel on a grid of 3 1000 Legendre nodes, that is, the grid will be composed of three grids:

- g1: 1000 points in 0, 1
- g2: 1000 points in 1, 0
- g3: 1000 points in 0, 00

You might want to only print the values of $V(k, k)$ for $k = k$ since the potential matrix is symmetric:

$$V(k, k) = V(k, k).$$

Benchmark: the resulting $V(k, k)$ on this grid can be benchmarked by comparing with the diagonal terms provided in `vkk_n3lo.dat`.

The **second step** will be to create a program and a module. The program will read an initialization file, e.g., `ini.dat` which will contain

```
2d0          #initial guess for mu
1d0          #amplitude of gap guess
0            # init_type: 0,1,2 (const., read)
1000        #n for space
1d0          #a for space
40d0        #b for space
400d0       #c for space
```

It will pass the space variables `n,a,b,c` to a subroutine `initialize` in the module which will create the Legendre grid and weights to be used in all integrations. Here you might want to remember that modules share variables with the main program that calls them. To avoid that and make sure that the grid you make is only used inside the module you want to declare it as

```
real(kind=8), allocatable, private, save :: ks(:), ws(:)
```

After making the grid, your function should read the potential `vn3lo.dat` in a 2-dimensional array, e.g., `v0`.

The **third step** will be to write a subroutine `solve_gap` that takes as input the chemical potential `mu`, the initialization type `init_type`, and the amplitude of the initial guess `d0` where we will be solving the gap equations. To begin, make an initial guess for the gap function, typically $\Delta^{(0)}(k) = d_0 = 1$, that is the amplitude of initial guess `d0` read from the initialization file. Make sure that this guess is controlled by an `if` statement using the `init_type`: later we will experiment with other ways of initializing the gap function, e.g., from the solution for a different `mu`. Integrate the guess on the RHS of the gap equation to get an updated guess. This will be done by writing the integral as

$$\int dp p^2 V(k, p) \frac{\Delta(p)}{E(p)} = 1 \quad \left[2 V(k, p) \frac{\Delta(p)}{E(p)} \right]$$

where w_p are the weights and nodes of the Legendre quadrature. Save the updated guess $\Delta^{(1)}(k)$ on a separate array and print and plot it.

The **fourth step** is to create a function that takes in the initial guess for the gap and the updated guess $\Delta^{(0)}(k)$, $\Delta^{(1)}(k)$ and returns the error

$$= \frac{\Delta^{(0)}(k) - \Delta^{(1)}(k)}{m\Delta^{(1)}(k)}$$

Then iterate the above procedure, i.e., integrate $\Delta^{(1)}(k)$ on the RHS of the gap equation to get an updated guess $\Delta^{(2)}(k)$, and so on, until convergence. We will call the calculation converged when will have dropped below some threshold `tol_for_iter`. This iteration is essentially performing a gradient descent for the energy. For each iteration print the iteration number, the max of $\Delta(k)$ and the error in a file, e.g., `i_dmax_err.dat`.

For the **fifth step**, we will collect observables. Once the calculation has converged, write a subroutine that calculates the density from the second gap equation using the converged gap function $\Delta(k)$. Make this subroutine also calculate $p(k)$ and $F(k)$ (see above), and $\bar{\Delta}_F$ and Δ_F . Note that since our calculation is done on a grid, some care is taken in calculating $\bar{\Delta}_F$ (you might want to also print $k_F = k - k_F$ where k is the Legendre node that yields the lowest k_F). Write $\Delta(k)$, $E(k)$, $p(k)$, and $F(k)$ in a file and return $\bar{\Delta}_F$, Δ_F . For $\Delta(k)$, $p(k)$, and $F(k)$ opt for the following format (note `k2 = k2`)

k	k2	Dk	Ek	pk	Fk
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.

From the calculated density find the Fermi momentum

$$k_F = (3\pi^2\rho)^{1/3}.$$

For the **sixth task**, we will organize and print the results: this is up to you. What we need is to have at a glance the chemical potential we started with, k_F , Δ_F , $\bar{\Delta}_F$.

Plotting and interpreting

Now we will plot and look at the results.

1. Plot the gap function $\Delta(k)$.
2. Plot the occupation probabilities and condensate amplitudes $p(k)$ and $F(k)$ in the same plot.
 1. What would those look like in the absence of pairing?
 2. Where does k_F fall on the x-axis
3. Plot $E(k)$ as a function of k and k^2 and look at its minimum.
 1. Are we actually finding the minimum
 2. How would this look like in the absence of pairing.

Solutions for multiple chemical potentials & the phase transition

As a result of the previous steps you now have a functioning solver of the gap equations for a given interaction and chemical potential. As you saw, the density comes out of the calculation, that is $\rho = \rho(\mu)$. To explore a density range one has to explore a range of chemical potentials.

The **first step** is to perform the above calculations for a range of chemical potentials $\mu = 2, 10, 1, 20, 2, 30$. How to implement this is up to you but the easiest way is to modify your main code that calls the subroutine solving the gap equations so that it does successive calls for a range of chemical potentials.

The **second step** is to plot the pairing gap Δ_F as a function of

1. μ
 2. k_F
- Can you predict where the phase transition to the normal-state is. How does that relate to the phase-shifts that we saw in Lecture 3?